

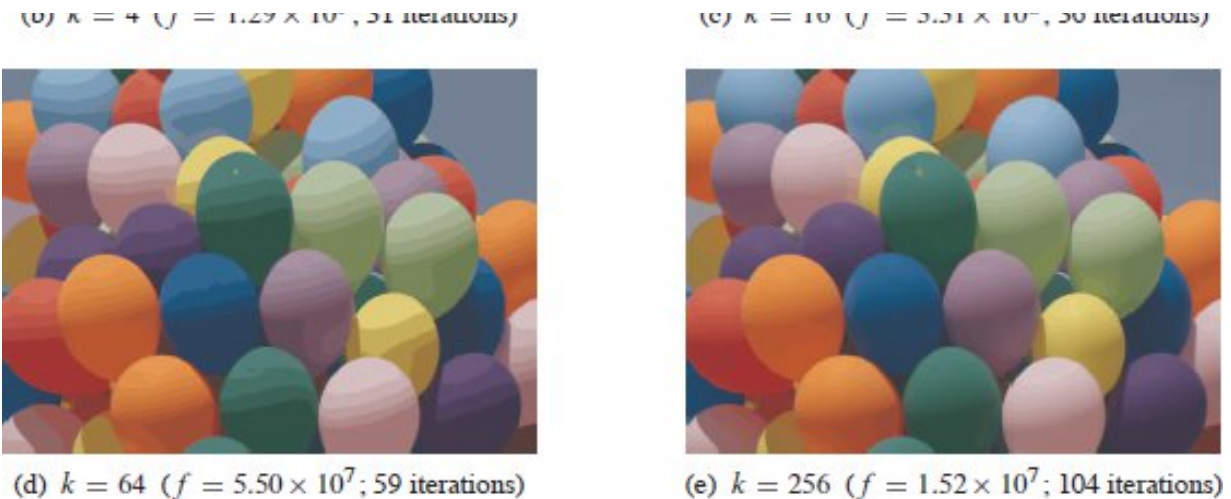


Figure 33.2 Using Lloyd’s procedure for vector quantization to compress a photo by using fewer colors. **(a)** The original photo has 350,000 pixels (700×500), each a 24-bit RGB (red/blue/green) triple of 8-bit values; these pixels (colors) are the “points” to be clustered. Points repeat, so there are only 79,083 distinct colors (less than 2^{24}). After compression, only k distinct colors are used, so each pixel is represented by only $\lceil \lg k \rceil$ bits instead of 24. A “palette” maps these values back to 24-bit RGB values (the cluster centers). **(b)–(e)** The same photo with $k = 4, 16, 64$, and 256 colors. (Photo from standuppaddle, pixabay.com.)

33.2 Multiplicative-weights algorithms

This section considers problems that require you to make a series of decisions. After each decision you receive feedback as to whether your decision was correct. We will study a class of algorithms that are called *multiplicative-weights algorithms*. This class of algorithms has a wide variety of applications, including game playing in economics, approximately solving linear-programming and multicommodity-flow problems, and various applications in online machine learning. We emphasize the online nature of the problem here: you have to make a sequence of decisions, but some of the information needed to make the i th decision appears only after you have already made the $(i - 1)$ st decision. In this section, we look at one particular problem, known as “learning from experts,” and develop an example of a multiplicative-weights algorithm, called the weighted-majority algorithm.

Suppose that a series of events will occur, and you want to make predictions about these events. For example, over a series of days, you

want to predict whether it is going to rain. Or perhaps you want to predict whether the price of a stock will increase or decrease. One way to approach this problem is to assemble a group of “experts” and use their collective wisdom in order to make good predictions. Let’s denote the experts, n of them, by $E_1, E_2, \dots, E_n$, and let’s say that T events are going to take place. Each event has an outcome of either 0 or 1, with $o^{(t)}$ denoting the outcome of the t th event. Before event t , each expert $E^{(i)}$ makes a prediction $q_i^{(t)} \in \{0, 1\}$. You, as the “learner,” then take the set of n expert predictions for event t and produce a single prediction $p^{(t)} \in \{0, 1\}$ of your own. You base your prediction only on the predictions of the experts and anything you have learned about the experts from their previous predictions. You do not use any additional information about the event. Only after making your prediction do you ascertain the outcome $o^{(t)}$ of event t . If your prediction $p^{(t)}$ matches $o^{(t)}$, then you were correct; otherwise, you made a mistake. The goal is to minimize the total number m of mistakes, where $m = \sum_{t=1}^T |p^{(t)} - o^{(t)}|$. You can also keep track of the number of mistakes each expert makes: expert E_i makes m_i mistakes, where $m_i = \sum_{t=1}^T |q_i^{(t)} - o^{(t)}|$.

For example, suppose that you are following the price of a stock, and each day you decide whether to invest in it for just that day by buying it at the beginning of the day and selling it at the end of the day. If, on some day, you buy the stock and it goes up, then you made the correct decision, but if the stock goes down, then you made a mistake. Similarly, if on some day, you do not buy the stock and it goes down, then you made the correct decision, but if the stock goes up, then you made a mistake. Since you would like to make as few mistakes as possible, you use the advice of the experts to make your decisions.

We’ll assume nothing about the movement of the stock. We’ll also assume nothing about the experts: the experts’ predictions could be correlated, they could be chosen to deceive you, or perhaps some are not really experts after all. What algorithm would you use?

Before designing an algorithm for this problem, we need to consider what is a fair way to evaluate our algorithm. It is reasonable to expect

that our algorithm performs better when the expert predictions are better, and that it performs worse when the expert predictions are worse. The goal of the algorithm is to limit the number of mistakes you make to be close to the number of mistakes that the best of the experts makes. At first, this goal might seem impossible, because you do not know until the end which expert is best. We'll see, however, that by taking the advice provided by all the experts into account, you can achieve this goal. More formally, we use the notion of "regret," which compares our algorithm to the performance of the best expert (in hindsight) over all. Letting $m^* = \min \{m_i : 1 \leq i \leq n\}$ denote the number of mistakes made by the best expert, the *regret* is $m - m^*$. The goal is to design an algorithm with low regret. (Regret can be negative, although it typically isn't, since it is rare that you do better than the best expert.)

As a warm-up, let's consider the case in which one of the experts makes a correct prediction each time. Even without knowing who that expert is, you can still achieve good results.

Lemma 33.3

Suppose that out of n experts, there is one who always makes the correct prediction for all T events. Then there is an algorithm that makes at most $\lceil \lg n \rceil$ mistakes.

Proof The algorithm maintains a set S consisting of experts who have not yet made a mistake. Initially, S contains all n experts. The algorithm's prediction is always the majority vote of the predictions of the experts remaining in set S . In case of a tie, the algorithm makes any prediction. After each outcome is learned, set S is updated to remove all the experts who made an incorrect prediction about that outcome.

We now analyze the algorithm. The expert who always makes the correct prediction will always be in set S . Every time the algorithm makes a mistake, at least half of the experts who were still in S also make a mistake, and these experts are removed from S . If S' is the set of experts remaining after removing those who made a mistake, we have that $|S'| \leq |S|/2$. The size of S can be halved at most $\lceil \lg n \rceil$ times until $|S| = 1$. From this point on, we know that the algorithm never makes a mistake, since the set S consists only of the one expert who never makes

a mistake. Therefore, overall the algorithm makes at most $\lceil \lg n \rceil$ mistakes.

■

Exercise 33.2-1 asks you to generalize this result to the case when there is no expert who makes perfect predictions and show that, for any set of experts, there is an algorithm that makes at most $m^* \lceil \lg n \rceil$ mistakes. The generalized algorithm begins in the same way. The set S might become empty at some point, however. If that ever happens, reset S to contain all the experts and continue the algorithm.

You can substantially improve your prediction ability by not just tracking which experts have not made any mistakes, or have not made any mistakes recently, to a more nuanced evaluation of the quality of each expert. The key idea is to use the feedback you receive to update your evaluation of how much trust to put in each expert. As the experts make predictions, you observe whether they were correct and decrease your confidence in the experts who make more mistakes. In this way, you can learn over time which experts are more reliable and which are less reliable, and weight their predictions accordingly. The change in weights is accomplished via multiplication, hence the term “multiplicative weights.”

The algorithm appears in the procedure **WEIGHTED-MAJORITY** on the following page, which takes a set $E = \{E_1, E_2, \dots, E_n\}$ of experts, a number T of events, the number n of experts, and a parameter $0 < \gamma \leq 1/2$ that controls how the weights change. The algorithm maintains weights $w_i^{(t)}$ for $i = 1, 2, \dots, n$ and $t = 1, 2, \dots, T$, where $0 < w_i^{(t)} \leq 1$. The **for** loop of lines 1–2 sets the initial weights $w_i^{(1)}$ to 1, capturing the idea that with no knowledge, you trust each expert equally. Each iteration of the main **for** loop of lines 3–18 does the following for an event $t = 1, 2, \dots, T$. Each expert E_i makes a prediction for event t in line 4. Lines 5–8 compute $upweight^{(t)}$, the sum of the weights of the experts who predict 1 for event t , and $downweight^{(t)}$, the sum of the weights of the experts who predict 0 for the event. Lines 9–11 decide the algorithm’s prediction $p^{(t)}$ for event t based on whichever weighted sum is larger (breaking ties in favor of deciding 1). The outcome of event t is revealed

in line 12. Finally, lines 14–17 decrease the weights of the experts who made an incorrect prediction for event t by multiplying their weights by $1 - \gamma$, leaving alone the weights of the experts who correctly predicted the event's outcome. Thus, the fewer mistakes each expert makes, the higher that expert's weight.

The WEIGHTED-MAJORITY procedure doesn't do much worse than any expert. In particular, it doesn't do much worse than the best expert. To quantify this claim, let $m^{(t)}$ be the number of mistakes made by the procedure through event t , and let $m_i^{(t)}$ be the number of mistakes made by expert E_i through event t . The following theorem is the key.

WEIGHTED-MAJORITY(E, T, n, γ)

```

1 for  $i = 1$  to  $n$ 
2    $w_i^{(1)} = 1$                                 // trust each expert equally
3 for  $t = 1$  to  $T$ 
4   each expert  $E_i \in E$  makes a prediction  $q_i^{(t)}$ 
5    $U = \{E_i : q_i^{(t)} = 1\}$                       // experts who predicted 1
6    $upweight^{(t)} = \sum_{i: E_i \in U} w_i^{(t)}$       // sum of weights of who predicted 1
7    $D = \{E_i : q_i^{(t)} = 0\}$                       // experts who predicted 0
8    $downweight^{(t)} = \sum_{i: E_i \in D} w_i^{(t)}$  // sum of weights of who predicted 0
9   if  $upweight^{(t)} \geq downweight^{(t)}$ 
10     $p^{(t)} = 1$                                 // algorithm predicts 1
11  else  $p^{(t)} = 0$                                 // algorithm predicts 0
12    outcome  $o^{(t)}$  is revealed
13    // If  $p^{(t)} \neq o^{(t)}$ , the algorithm made a mistake.
14    for  $i = 1$  to  $n$ 
15      if  $q_i^{(t)} \neq o^{(t)}$                         // if expert  $E^{(i)}$  made a mistake ...
16         $w_i^{(t+1)} = (1 - \gamma)w_i^{(t)}$           // ... then decrease that expert's weight
17      else  $w_i^{(t+1)} = w_i^{(t)}$ 
18    return  $p^{(t)}$ 

```

Theorem 33.4

When running WEIGHTED-MAJORITY, we have, for every expert E_i and every event $T \leq T$,

$$m^{(T')} \leq 2(1 + \gamma)m_i^{(T')} + \frac{2 \ln n}{\gamma} . \quad (33.5)$$

Proof Every time an expert E_i makes a mistake, its weight, which is initially 1, is multiplied by $1 - \gamma$, and so we have

$$w_i^{(t)} = (1 - \gamma)^{m_i^{(t)}} \quad (33.6)$$

for $t = 1, 2, \dots, T$.

We use a potential function $W(t) = \sum_{i=1}^n w_i^{(t)}$, summing the weights for all n experts after iteration t of the **for** loop of lines 3–18. Initially, we have $W(0) = n$ since all n weights start out with the value 1. Because each expert belongs to either the set U or the set D (defined in lines 5 and 7 of WEIGHTED-MAJORITY), we always have $W(t) = \text{upweight}^{(t)} + \text{downweight}^{(t)}$ after each execution of line 8.

Consider an iteration t in which the algorithm makes a mistake in its prediction, which means that either the algorithm predicts 1 and the outcome is 0 or the algorithm predicts 0 and the outcome is 1. Without loss of generality, assume that the algorithm predicts 1 and the outcome is 0. The algorithm predicted 1 because $\text{upweight}^{(t)} \geq \text{downweight}^{(t)}$ in line 9, which implies that

$$\text{upweight}^{(t)} \geq W(t)/2 . \quad (33.7)$$

Each expert in U then has its weight multiplied by $1 - \gamma$, and each expert in D has its weight unchanged. Thus, we have

$$\begin{aligned} W(t+1) &= \text{upweight}^{(t)}(1 - \gamma) + \text{downweight}^{(t)} \\ &= \text{upweight}^{(t)} + \text{downweight}^{(t)} - \gamma \cdot \text{upweight}^{(t)} \\ &= W(t) - \gamma \cdot \text{upweight}^{(t)} \\ &\leq W(t) - \gamma \frac{W(t)}{2} \quad (\text{by inequality (33.7)}) \\ &= W(t)(1 - \gamma/2) . \end{aligned}$$

Therefore, for every iteration t in which the algorithm makes a mistake, we have

$$W(t + 1) \leq (1 - \gamma/2)W(t) . \quad (33.8)$$

In an iteration where the algorithm does not make a mistake, some of the weights decrease and some remain unchanged, so that we have

$$W(t + 1) \leq W(t) . \quad (33.9)$$

Since there are $m^{(T')}$ mistakes made through iteration T' , and $W(1) = n$, we can repeatedly apply inequality (33.8) to iterations where the algorithm makes a mistake and inequality (33.9) to iterations where the algorithm does not make a mistake, obtaining

$$W(T') \leq n(1 - \gamma/2)^{m^{(T')}} . \quad (33.10)$$

Because the function W is the sum of the weights and all weights are positive, its value exceeds any single weight. Therefore, using equation (33.6) we have, for any expert E_i and for any iteration $T' \leq T$,

$$W(T') \geq w_i^{(T')} = (1 - \gamma)^{m_i^{(T')}} . \quad (33.11)$$

Combining inequalities (33.10) and (33.11) gives

$$(1 - \gamma)^{m_i^{(T')}} \leq n(1 - \gamma/2)^{m^{(T')}} .$$

Taking the natural logarithm of both sides yields

$$m_i^{(T')} \ln(1 - \gamma) \leq m^{(T')} \ln(1 - \gamma/2) + \ln n . \quad (33.12)$$

We now use the Taylor series expansion to derive upper and lower bounds on the logarithmic factors in inequality (33.12). The Taylor series for $\ln(1+x)$ is given in equation (3.22) on page 67. Substituting $-x$ for x , we have that for $0 < x \leq 1/2$,

$$\ln(1 - x) = -x - \frac{x^2}{2} - \frac{x^3}{3} - \frac{x^4}{4} - \dots . \quad (33.13)$$

Since each term on the right-hand side is negative, we can drop all terms except the first and obtain an upper bound of $\ln(1 - x) \leq -x$. Since $0 < \gamma \leq 1/2$, we have

$$\ln(1 - \gamma/2) \leq -\gamma/2 . \quad (33.14)$$

For the lower bound, Exercise 33.2-2 asks you to show that $\ln(1 - x) \geq -x - x^2$ when $0 < x \leq 1/2$, so that

$$-\gamma - \gamma^2 \leq \ln(1 - \gamma) . \quad (33.15)$$

Thus, we have

$$\begin{aligned} m_i^{(T')}(-\gamma - \gamma^2) &\leq m_i^{(T')} \ln(1 - \gamma) && \text{(by inequality (33.15))} \\ &\leq m_i^{(T')} \ln(1 - \gamma/2) + \ln n && \text{(by inequality (33.12))} \\ &\leq m_i^{(T')}(-\gamma/2) + \ln n && \text{(by inequality (33.14)) ,} \end{aligned}$$

so that

$$m_i^{(T')}(-\gamma - \gamma^2) \leq m_i^{(T')}(-\gamma/2) + \ln n . \quad (33.16)$$

Subtracting $\ln n$ from both sides of inequality (33.16) and then multiplying both sides by $-2/\gamma$ yields $m_i^{(T')} \leq 2(1 + \gamma)m_i^{(T')} + (2\ln n)/\gamma$, thus proving the theorem. ■

Theorem 33.4 applies to any expert and any event $T \leq T$. In particular, we can compare against the best expert after all events have occurred, producing the following corollary.

Corollary 33.5

At the end of procedure WEIGHTED-MAJORITY, we have

$$m^{(T)} \leq 2(1 + \gamma)m^* + \frac{2\ln n}{\gamma} . \quad (33.17)$$

■

Let's explore this bound. Assuming that $\sqrt{\ln n/m^*} \leq 1/2$, we can choose $\gamma = \sqrt{\ln n/m^*}$ and plug into inequality (33.17) to obtain

$$\begin{aligned} m^{(T)} &\leq 2 \left(1 + \sqrt{\frac{\ln n}{m^*}} \right) m^* + \frac{2\ln n}{\sqrt{\ln n/m^*}} \\ &= 2m^* + 2\sqrt{m^* \ln n} + 2\sqrt{m^* \ln n} \\ &= 2m^* + 4\sqrt{m^* \ln n} , \end{aligned}$$

and so the number of errors is at most twice the number of errors made by the best expert plus a term that is often slower growing than m^* . Exercise 33.2-4 shows that you can decrease the bound on the number of errors by a factor of 2 by using randomization, which leads to much stronger bounds. In particular, the upper bound on regret ($m - m^*$) is reduced from $(1 + 2\gamma)m^* + (2 \ln n)/\gamma$ to an expected value of $\epsilon m^* + (\ln n)/\epsilon$, where both γ and ϵ are at most $1/2$. Numerically, we can see that if $\gamma = 1/2$, WEIGHTED-MAJORITY makes at most 3 times the number of errors as the best expert, plus $4 \ln n$ errors. As another example, suppose that $T = 1000$ predictions are being made by $n = 20$ experts, and the best expert is correct 95% of the time, making 50 errors. Then WEIGHTED-MAJORITY makes at most $100(1+\gamma)+2 \ln 20/\gamma$ errors. By choosing $\gamma = 1/4$, WEIGHTED-MAJORITY makes at most 149 errors, or a success rate of at least 85%.

Multiplicative weights methods typically refer to a broader class of algorithms that includes WEIGHTED-MAJORITY. The outcomes and predictions need not be only 0 or 1, but can be real numbers, and there can be a loss associated with a particular outcome and prediction. The weights can be updated by a multiplicative factor that depends on the loss, and the algorithm can, given a set of weights, treat them as a distribution on experts and use them to choose an expert to follow in each event. Even in these more general settings, bounds similar to Theorem 33.4 hold.

Exercises

33.2-1

The proof of Lemma 33.3 assumes that some expert never makes a mistake. It is possible to generalize the algorithm and analysis to remove this assumption. The new algorithm begins in the same way. The set S might become empty at some point, however. If that ever happens, reset S to contain all the experts and continue the algorithm. Show that the number of mistakes that this algorithm makes is at most $m^* \lceil \lg n \rceil$.

33.2-2

Show that $\ln(1 - x) \geq -x - x^2$ when $0 < x \leq 1/2$. (*Hint*: Start with equation (33.13), group all the terms after the first three, and use equation (A.7) on page 1142.)

33.2-3

Consider a randomized variant of the algorithm given in the proof of Lemma 33.3, in which some expert never makes a mistake. At each step, choose an expert E_i uniformly at random from the set S and then make the same predication as E_i . Show that the expected number of mistakes made by this algorithm is $\lceil \lg n \rceil$.

33.2-4

Consider a randomized version of WEIGHTED-MAJORITY. The algorithm is the same, except for the prediction step, which interprets the weights as a probability distribution over the experts and chooses an expert E_i according to that distribution. It then chooses its prediction to be the same as the prediction made by expert E_i . Show that, for any $0 < \epsilon < 1/2$, the expected number of mistakes made by this algorithm is at most $(1 + \epsilon)m^* + (\ln n)/\epsilon$.

33.3 Gradient descent

Suppose that you have a set $\{p_1, p_2, \dots, p_n\}$ of points and you want to find the line that best fits these points. For any line ℓ , there is a distance d_i between each point p_i and the line. You want to find the line that minimizes some function $f(d_1, \dots, d_n)$. There are many possible choices for the definition of distance and for the function f . For example, the distance can be the projection distance to the line and the function can be the sum of the squares of the distances. This type of problem is common in data science and machine learning—the line is the hypothesis that best describes the data—where the particular definition of best is determined by the definition of distance and the objective f . If the definition of distance and the function f are linear, then we have a linear-programming problem, as discussed in Chapter 29. Although the